

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA1202	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. DMA-Based Data Transfer from I/O Device to Memory

Aim:

To design a DMA-based data transfer algorithm that transfers data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Pseudocode:

ALGORITHM DMA_Data_Transfer

INPUT:

I/O_Device

Memory_Address

Data_Size

OUTPUT:

Data transferred from I/O device to memory

BEGIN

Initialize DMA_Controller

Set DMA_Source $\leftarrow$ I/O_Device

Set DMA_Destination $\leftarrow$ Memory_Address

Set DMA_Count $\leftarrow$ Data_Size

Enable DMA_Interrupt

Request_DMA_Control()

IF DMA_Control_Granted = TRUE THEN

Start_DMA_Transfer()

WHILE DMA_Count > 0 DO

Transfer_Data_From_IO_To_Memory()

Update_Memory_Address()

Decrease_DMA_Count()

END WHILE

Generate_DMA_Interrupt()

END IF

IF DMA_Interrupt_Received = TRUE THEN

 Acknowledge_Interrupt()

 Check_Transfer_Status()

 IF Transfer_Status = SUCCESS THEN

 Display "DMA Transfer Completed"

 ELSE

 Display "DMA Transfer Error"

 END IF

 Release_DMA_Control()

END IF

END

Result:

The DMA controller transfers data directly from the I/O device to memory and interrupts the CPU only after the transfer is completed, thereby minimizing CPU involvement.

2. Vectored Interrupt Handling System

Aim:

To develop an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches the appropriate service routines.

Pseudocode:

ALGORITHM Vectored_Interrupt_Handler

INPUT:

 Interrupts from multiple devices

OUTPUT:

 Appropriate Interrupt Service Routine executed

BEGIN

Initialize Interrupt_Controller

Define Interrupt_Priority

WHILE System_Running = TRUE DO

Check_For_Interrupts()

IF Interrupt_Exists = TRUE THEN

Identify_Interrupt_Vector()

Determine_Highest_Priority_Interrupt()

Save_CPU_Context()

Acknowledge_Highest_Priority_Interrupt()

SWITCH Interrupt_Vector

CASE TIMER_INTERRUPT:

Call Timer_ISR()

CASE KEYBOARD_INTERRUPT:

Call Keyboard_ISR()

CASE DISK_INTERRUPT:

Call Disk_ISR()

CASE NETWORK_INTERRUPT:

Call Network_ISR()

DEFAULT:

Call Default_ISR()

END SWITCH

Restore_CPU_Context()

Return_From_Interrupt()

END IF

END WHILE

END

Result:

The system identifies the highest-priority vectored interrupt and directly dispatches its corresponding Interrupt Service Routine, ensuring efficient interrupt handling.

3. Dynamic Selection of Memory-Mapped I/O and I/O-Mapped I/O

Aim:

To dynamically select between memory-mapped I/O and I/O-mapped I/O based on latency and bandwidth requirements.

Pseudocode:

ALGORITHM Select_IO_Mapping

INPUT:

Latency_Requirement

Bandwidth_Requirement

OUTPUT:

Selected I/O Mapping Technique

BEGIN

Define LATENCY_THRESHOLD

Define BANDWIDTH_THRESHOLD

IF Latency_Requirement <= LATENCY_THRESHOLD AND
Bandwidth_Requirement >= BANDWIDTH_THRESHOLD THEN

Select MEMORY_MAPPED_IO

Display "Memory-Mapped I/O Selected"

ELSE IF Latency_Requirement > LATENCY_THRESHOLD AND
Bandwidth_Requirement < BANDWIDTH_THRESHOLD THEN

Select IO_MAPPED_IO

Display "I/O-Mapped I/O Selected"

ELSE

IF Bandwidth_Requirement >= BANDWIDTH_THRESHOLD THEN

Select MEMORY_MAPPED_IO

Display "Memory-Mapped I/O Selected"

ELSE

Select IO_MAPPED_IO

Display "I/O-Mapped I/O Selected"

END IF

END IF

Configure_Selected_IO_Method()

END

Result:

The algorithm selects an appropriate I/O mapping technique according to the required latency and bandwidth, allowing flexible I/O communication.

4. Bus Arbitration for PCIe, USB and Thunderbolt**Aim:**

To design a bus arbitration algorithm that manages multiple devices communicating through PCIe, USB, and Thunderbolt interfaces.

Pseudocode:

ALGORITHM Bus_Arbitration

INPUT:

Requests from PCIe, USB and Thunderbolt devices

OUTPUT:

Bus access granted to selected device

BEGIN

Initialize Bus_Arbiter

WHILE System_Running = TRUE DO

 Check_Bus_Requests()

 IF PCIe_Request = TRUE THEN

 Add PCIe to Request_Queue

 END IF

 IF USB_Request = TRUE THEN

 Add USB to Request_Queue

 END IF

```

IF Thunderbolt_Request = TRUE THEN
    Add Thunderbolt to Request_Queue
END IF

IF Request_Queue is NOT EMPTY THEN

    Calculate_Priority_For_Each_Request()

    Select_Highest_Priority_Request()

    Grant_Bus_Access(Selected_Device)

    WHILE Selected_Device_Has_Data = TRUE DO
        Transfer_Data()
    END WHILE

    Release_Bus()

    Remove Selected_Device from Request_Queue

END IF

END WHILE

```

END

Result:

The bus arbiter manages requests from PCIe, USB, and Thunderbolt devices by prioritizing requests and granting bus access efficiently to reduce communication conflicts.

5. Hybrid I/O Communication Using Polling and DMA

Aim:

To implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Pseudocode:

ALGORITHM Hybrid_IO_Communication

INPUT:

I/O_Device

Device_Speed

Data_Size

OUTPUT:

Data transferred using appropriate I/O method

BEGIN

Define SPEED_THRESHOLD

IF Device_Speed < SPEED_THRESHOLD THEN

Display "Low-Speed Device: Polling Selected"

WHILE Data_Available = FALSE DO

Poll_Device_Status()

END WHILE

Read_Data_From_Device()

Store_Data_In_Memory()

Display "Polling Transfer Completed"

ELSE

Display "High-Speed Device: DMA with Interrupt Selected"

Initialize_DMA_Controller()

Set DMA_Source $\leftarrow$ I/O_Device

Set DMA_Destination $\leftarrow$ Memory

Set DMA_Count $\leftarrow$ Data_Size

Enable_DMA_Interrupt()

Start_DMA_Transfer()

WHILE DMA_Transfer_Completed = FALSE DO

 CPU_Performs_Other_Tasks()

END WHILE

IF DMA_Interrupt_Received = TRUE THEN

 Acknowledge_Interrupt()

 Check_Transfer_Status()

 IF Transfer_Status = SUCCESS THEN

 Display "DMA Transfer Completed"

 ELSE

 Display "DMA Transfer Error"

 END IF

 END IF

END IF

END

Result:

The hybrid mechanism uses polling for low-speed devices and DMA with interrupts for high-speed devices, reducing unnecessary CPU involvement while providing efficient I/O communication.

CONCLUSION

The five algorithms demonstrate important I/O communication mechanisms in computer architecture. DMA enables direct data transfer with minimal CPU involvement, vectored interrupts provide efficient priority-based interrupt handling, and dynamic I/O mapping selects a suitable technique based on requirements. Bus arbitration manages multiple high-speed interfaces, while hybrid I/O combines polling and DMA with interrupts to improve overall communication efficiency.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient